From Code Repository to Research Infrastructure: Evaluating GitHub for Managing the Scientific Research Lifecycle

Telmo Miguel-Medina[1*]

[1]Electromechanical Engineering Department, University of Burgos, Burgos, Spain.

Corresponding author(s). E-mail(s): tmiguel@ubu.es;

Abstract

Aims: Research artifacts are usually managed across disconnected tools, with weak traceability of the process that produced the outputs. We ask to what extent GitHub's native functionalities can serve as an integrated infrastructure for managing the scientific research process across the lifecycle.

Methods: A hybrid design: a bibliometric analysis of a 5,139-work OpenAlex and arXiv corpus (2008–2025) mapped onto lifecycle stages; a structured review of 67 studies yielding fifteen research management requirements; a reproducible four-level assessment of GitHub's 68 native capabilities against them; a reference architecture and reusable template synthesised from the mapping; and a self-referential implementation, evaluated on six dimensions and re-instantiated on a second, unrelated project.

Results: The literature is output-biased — data management and analysis dominate (50.8% and 41.9% of the corpus) against 3.7% for idea and question formulation. GitHub supports 13 of 15 requirements at partial or better and 7 directly (mean 2.43 of 3 for the fourteen core requirements), none unsupported, but support is direct where research resembles software engineering and only partial or limited for the research-specific traceability of questions, decisions and provenance — the requirements the literature also under-serves (1.67 versus 2.50). Five requirements need external infrastructure. A 15-component architecture with four mandatory conventions and a 33-file template operationalise what is covered; in the self-referential implementation 13 of 15 components were exercised and the six-dimension score reached 1.86 of 2 (usability lowest at 1.50), and a retrospective instantiation on an unrelated clinical-machine-learning project reproduced the profile.

Conclusion: GitHub can serve as an integrated coordination and traceability *layer* for a substantial part of the research lifecycle — strong from execution to output, weak upstream, and dependent on complementary infrastructure for preservation and large data. It maps the research-process-management gap rather than closing it.

1 Introduction

1.1 Complexity and artifact proliferation in modern research

A contemporary research project is a chain of interdependent objects: a research question, a protocol, a literature corpus, data, analysis code, a computational environment, intermediate results, figures, tables, a manuscript, a publication, released software and supplementary material. These objects evolve continuously and are produced by distributed, often multidisciplinary teams. Computational and data-intensive research has amplified both the number of artifacts and the need to keep them consistent with one another [1–3].

1.2 Fragmentation of research-process management

In common practice these artifacts and the activities that generate them are spread across disconnected systems: email, cloud storage, local folders, spreadsheets, reference managers, statistical software, code repositories, task managers and manuscript editors. Each tool serves its local function, but the coordination layer between them is largely absent. Reviews of research data management, scholarly infrastructure and post-award reporting repeatedly associate this fragmentation with weak decision traceability, incomplete process documentation, inconsistent versioning, limited provenance and low project visibility [4–7].

1.3 Managing the process, not only archiving the outputs

Two decades of work on scientific workflow systems, provenance, research data management, virtual research environments and open-science platforms have produced mature evidence on *output* stewardship — repositories, persistent identifiers, curation, dissemination [8–12]. The management of the research *process* that produces those outputs — planning, question formulation, decision tracking, cross-stage coordination — has received comparatively little systematic, quantitative attention [13–15].

1.4 Prior reviews, by strand

The evidence base is a set of partially connected review literatures. *Scientific workflow systems* are the most surveyed: recurring reviews catalogue execution engines, scheduling and data-intensive orchestration [1–3], but they take the research question, the analysis plan and the decisions behind a pipeline as given and start at the executable graph. *Provenance* research extends this with models for capturing and

querying the lineage of workflow runs [8, 9, 16]; systematic reviews of provenance systems repeatedly note that coverage stops at data and computation and rarely reaches the upstream intellectual choices. *Research data management* reviews document institutional services, roles and lifecycle models for curating and sharing data [4, 5, 10, 13], again concentrating on data as an output rather than on the process that generates it. *Research information systems* and knowledge-graph reviews describe institutional infrastructures for recording researchers, projects and outputs for reporting and evaluation [12, 17], at an administrative rather than a project-working granularity. *Open-science* syntheses map practices and incentives for transparency, preprints and reproducible reporting [11, 18]. Finally, a *version-control-in-research* strand — mostly guidance and case reports rather than reviews — argues that Git and GitHub can carry collaborative, reproducible research [19–22]. This study cuts across these strands: it treats the research *process* as the object of management and asks how far one widely available platform can serve the requirements they each touch but none addresses as a whole.

1.5 Code-hosting platforms beyond software

GitHub already provides primitives for coordination, versioning, issue tracking, structured discussion, review and automation. Guidance and case studies describe its use for versioned, collaborative and reproducible research in computational biology [19, 20, 23], ecology and evolution [22], laboratory research [24], and for developing community data standards [25]; its footprint in the scholarly record is itself growing and measurable [26]. Yet this evidence is case-based, and studies that mine GitHub as research data caution that its use is heterogeneous and often shallow [27, 28]. Whether GitHub's primitives can be organised into an infrastructure for managing the research process, and which lifecycle stages such an infrastructure can actually cover, has not been established on an evidentiary basis.

1.6 Aim and research questions

The aim of this study is to characterise the literature on digital infrastructure for research-process management, derive a lifecycle-structured set of research management requirements, determine quantitatively how far GitHub's native functionalities cover them, consolidate the covered functionalities into a reusable reference architecture and template, and probe feasibility through a self-referential implementation. The study does not claim that GitHub replaces specialised research infrastructure.

RQ1 How has the literature on digital infrastructure for managing the research process evolved in volume, disciplines, venues and thematic structure, and which lifecycle stages are under-represented?
RQ2 What research project management requirements across the research lifecycle can be derived from this literature?
RQ3 To what extent, with what support level and limitations, do GitHub's native functionalities cover these requirements, and how is coverage distributed across lifecycle stages?

RQ4 How can the covered functionalities be organised into a coherent reusable reference architecture and project template?
RQ5 What does a self-referential implementation, together with the reusable template, reveal about feasibility, traceability gains and limitations relative to a fragmented workflow?

2 Materials and Methods

2.1 Overall design

The study is a multi-method evidence synthesis with three coupled components: a bibliometric analysis of the field (RQ1); a structured requirement extraction and a reproducible requirement–functionality coverage analysis (RQ2, RQ3); and a design synthesis with a single self-referential feasibility test (RQ4, RQ5). All datasets, coding sheets, mapping tables and analysis scripts are openly available (Section 5); every table and figure in this paper is regenerated by a versioned script from those inputs.

2.2 Bibliometric corpus and analysis (RQ1)

The primary corpus was built from OpenAlex, which indexes journal articles, conference papers and preprints with abstracts, concepts and citations and is fully reproducible from a published query. Fifteen `title_and_abstract` queries were run, grouped in four strands: core process management (*"research project management"*, *"research workflow"*, *"research lifecycle"*, *"scientific workflow"* and near-variants), data management and open science (*"research data management"*, *"open science"*, *"reproducible research"*), version control in research (*github*, *"version control"* + *research*), and research environments (*"virtual research environment"*, *"science gateway"*, *"research information system"*). All were filtered to `2008--2025` and to document types article, review and proceedings-article, returning 6,512 hits and 5,936 unique works. Five narrower abstract- and title-scoped arXiv queries (*"version control"*, *github* within `cs.DL/SE/CY/DC`, *"scientific workflow"*, *"research software"*, *"reproducible research"*) added 462 preprints, for 6,398 input records. De-duplication removed 1,167 data- and software-repository deposits (Zenodo, Figshare and similar, by an explicit source list), 29 duplicate DOIs, 9 duplicate title–year pairs and 31 no-DOI rows matching a DOI row on title and year, yielding a corpus of **5,139 works** (OpenAlex 4,689, arXiv 426, both 24).

As an independent coverage check, the same four query strands were run in Scopus (`TITLE-ABS-KEY`, document types article/review/conference paper) and Web of Science (`TS=` Topic), returning 18,250 raw and 13,586 de-duplicated in-window records. These share **22% of their DOIs** with the OpenAlex/arXiv corpus; inspection of the non-overlapping set shows it is dominated by recent bioinformatics and computer-science tool papers that cite a GitHub repository and mention "workflow" or "reproducibility" in passing — the low-precision tail a title- and abstract-anchored search deliberately excludes — rather than missed core literature on research-process infrastructure. A heuristic precision estimate for the broadest OpenAlex queries was

≈ 0.94. OpenAlex plus arXiv was therefore retained as the primary corpus for openness and full query reproducibility; the raw Scopus and Web of Science exports are archived with the dataset record.

Analysis covered annual production, source and disciplinary profile, country distribution, and a keyword co-occurrence map. Corpus concepts and abstracts were then matched against an eleven-item lexicon of research-lifecycle stages (idea, question, planning, literature, methods, data, analysis, results, dissemination, outputs, governance) to produce a stage-hit profile. Because indexing of 2025 was incomplete at retrieval, growth statements use a 2008–2024 series.

2.3 Structured review and requirement extraction (RQ2)

The requirement extraction rests on a structured evidence base of **52 review, survey and synthesis papers** identified through a deep-search synthesis prioritising secondary literature, complemented by **15 purposively selected primary studies** of Git and GitHub use in research (case reports and mining studies spanning computational biology, ecology, geography and software-repository research). From this base, each documented literature challenge was traced to a management need and then to a derived requirement following the chain *finding* → *problem* → *need* → *requirement* → *category*, with the supporting studies recorded per row. Seventeen requirement rows were extracted and consolidated, by thematic grouping and reconciliation with a provisional domain list, into a framework of fifteen requirements (RM1–RM15), each with a definition, expected capabilities, lifecycle stages, evidence, a category, a relative importance and its interdependencies with the others. Requirements rated of high importance but weak in literature attention were flagged as *differentiators*. Extraction was performed by a single coder. To probe coding stability, all coded decisions in the study — the seventeen requirement-row assignments, the fifteen support levels of Section 2.4 and the twenty-one evaluation sub-scores of Section 2.7 — were subsequently re-coded independently from the same source material and rubric; exact agreement was 96% (Cohen's κ 0.83–1.00 by unit), and the two adjacent-category disagreements leave every figure reported here unchanged. The absence of a second *human* coder remains a limitation (Section 4.6).

2.4 GitHub capability catalogue and mapping rubric (RQ3)

GitHub's *native* functionality was catalogued as **68 capabilities in twelve feature groups** (repositories and Git, Markdown and documentation, Issues, Projects, Milestones, Labels, Discussions, Branches, Pull Requests, Actions, Releases and tags, and access/identity). Marketplace and third-party applications were excluded; the one retained non-native item is the GitHub–Zenodo release webhook, the de facto standard bridge for persistent identification. Each capability records its function, research use, the durable record it leaves, its plan availability (verified against GitHub documentation) and its limitations.

Each requirement RM1–RM15 was then assigned the best support level GitHub's native functionality can reach with a reasonable implementation pattern, using a four-level rubric: *Direct* (3) — a native feature meets the requirement; *Partial* (2)

— met with an additional process or convention on native features; *Limited* (1) — marginal support only; *Not supported* (0). Every assignment carries an evidence note, the primary capabilities, an implementation pattern, and any external-tool dependency. A stricter variant capping convention-heavy support at Limited was scored as a robustness check.

2.5 Coverage indicators (RQ3)

From the mapping we computed the support distribution, the mean support overall and by requirement category, the mean for the differentiators versus the rest, the count of requirements needing external tools, and a lifecycle-coverage profile (support weighted by each requirement's applicability to each stage). Because one requirement (governance and sustainability) is only partly in scope, the headline mean is reported for the fourteen core requirements as well as for all fifteen.

2.6 Reference architecture and template (RQ4)

The Direct- and Partial-support capabilities were synthesised into a layered reference architecture of named components, each a single GitHub configuration serving a set of requirements, together with the conventions that make generic features research-usable, a lifecycle model and seven workflow definitions. The architecture was operationalised as a discipline-independent GitHub template repository, with a manifest mapping every file to the components and requirements it serves.

2.7 Self-referential implementation and evaluation (RQ5)

Following a self-referential design, the framework was applied to the development of this study; the repository is the evidence. The documentation, decision-log and version-control components were exercised natively throughout Phases 1–11. The publication phase then enacted the parts a single-author sprint had left configured: the repository was made public, and the study's own outstanding work was reorganised into twelve GitHub Issues on the configured Project and taken through one full Issue → branch → pull request → merge → Release cycle, moving the coordination layer from configured to exercised once. Each of the fifteen architecture components was then classified by implementation status (native, partial or planned). The framework was evaluated on six dimensions — requirement coverage, traceability, documentation, organization, transparency, usability — with 21 sub-scores on a 0/1/2 scale, each tagged by basis as observed, retrospective, design-support or planned; after the publication phase, 20 of the 21 are observed. As external-validity evidence, the template was additionally instantiated retrospectively on a completed, published clinical-machine-learning project in a different discipline, mapping each of RM1–RM15 onto a template mechanism. A conventional fragmented workflow was contrasted with the framework across six traceability and visibility dimensions.

3 Results

3.1 Bibliometric map of the field (RQ1)

The literature is an assembly of partially connected strands — scientific workflow systems, research data management, provenance, research information systems, open science and version control — rather than one field. In the 5,139-work corpus, annual output rises nearly tenfold between 2008 and 2024. Publications concentrate in computer science, library and information science and the biomedical literature; the leading venues after removing repository deposits are the *International Journal of Digital Curation*, *Concurrency and Computation*, *Future Generation Computer Systems*, *F1000Research* and *PLoS ONE*, and the leading countries are the United States, Germany, the United Kingdom, China and the Netherlands. The keyword co-occurrence map (Figure 1) shows the corpus spanning computer science, data science, knowledge management and the social sciences, with workflow, provenance and research data management as recurring bridging terms.

The stage-hit profile is markedly uneven (Table 1). Data management and analysis dominate (50.8% and 41.9% of the corpus); dissemination and coordination follow (24.4% and 21.1%); and idea and question formulation are at the bottom (3.7%), with provenance at 7.0%. A stage-level reading rates seven of fifteen lifecycle stages as thinly covered: idea, question, planning, ethics review, post-publication traceability, cross-stage coordination and governance. The structured review of 52 secondary studies reproduces the same bias independently.

Table 1 Research-lifecycle stage coverage in the bibliometric corpus (RQ1). Share of the 5,139-work corpus whose concepts or abstract match the stage lexicon.

Stage	% of corpus	Stage	% of corpus
Data management	50.8	Planning	12.7
Analysis / workflow	41.9	Literature	10.0
Dissemination	24.4	Outputs / identification	11.9
Coordination	21.1	Governance	10.9
Methods / protocol	13.7	Provenance	7.0
		Idea / question	**3.7**

3.2 Requirements and GitHub coverage (RQ2, RQ3)

Requirements (RQ2).

Fifteen requirements were derived (Table 2), concentrated in traceability (four), documentation (three) and collaboration (two). Three requirements — research planning (RM1), research-question management (RM2) and decision traceability (RM5) — are of high importance yet weakly attended to in the literature; these differentiators define the gap the study targets. The requirements are interdependent: provenance (RM10)

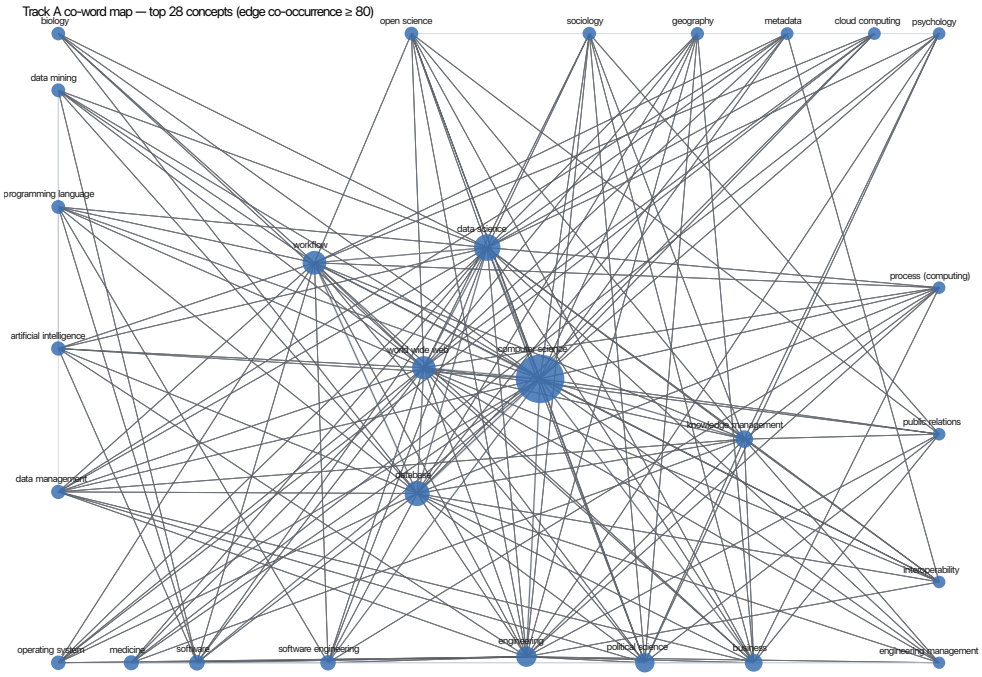

Fig. 1 Keyword co-occurrence map of the 5,139-work corpus (top concepts; edges are strong co-occurrences). The field spans computer science, data science, information management and the social sciences, bridged by workflow, provenance and research data management.

consumes RM2, RM4, RM5, RM8 and RM9, and transparency (RM11) is an emergent property of RM4, RM5, RM8 and RM10.

Coverage (RQ3).

GitHub supports **13 of 15 requirements at Partial or better and 7 at Direct**, with none unsupported. The mean support is **2.43 of 3** for the fourteen core requirements (2.33 for all fifteen). Support is Direct where research work resembles software engineering — task management, review-based collaboration, communication, version control, documentation, transparency and automation, all with category means of 3.0 — and Partial or Limited for research-specific traceability (RM2 Limited; RM5 and RM10 Partial), planning (RM1 Partial) and governance (RM15 Limited) (Table 3). The differentiator requirements have a mean support of **1.67 against 2.50** for the rest: the upstream gap seen bibliometrically in RQ1 reappears, independently, in GitHub's coverage.

Five requirements cannot be met by GitHub alone — version control of large data (RM8), integration across heterogeneous tools (RM9), reproducibility (RM12), persistent identification and preservation (RM14) and governance (RM15) — and require external infrastructure. The lifecycle-coverage profile is comparatively flat (weighted

mean support 2.1–2.4 per stage) but lowest at idea (2.12) and question (2.25) and highest at literature, analysis and manuscript, echoing the RQ1 shape. Issues and the Git repository carry most of the framework; Projects, Actions, Discussions, Branches and Releases each contribute to only a few requirements. Under the stricter rubric the overall mean falls to 2.13, but the distribution (7 Direct, 3 Partial, 5 Limited, 0 unsupported) and the qualitative conclusion are unchanged.

By requirement.

The seven Direct requirements are GitHub's software-engineering home ground: task management (RM3) via issues, sub-issues, a board with status and priority, and closing keywords that bind a task to the change that resolved it; documentation (RM4) as version-controlled Markdown written from project start; collaboration (RM6) and communication (RM7) via pull-request review, suggested changes, `CODEOWNERS` and categorised Discussions; version control (RM8) as the platform substrate of history, diff, restore and tags; transparency (RM11) as an inherent property of a public repository; and automation (RM13) through event-, schedule- and dispatch-triggered Actions. The six Partial requirements are met only by imposing a convention on a generic feature: research planning (RM1) as a Project with a "research phase" field and a roadmap view but no plan object and no plan-revision history; decision traceability (RM5) as a decision-record issue template or a versioned log, which works — and is demonstrated by this repository — but which GitHub cannot distinguish from ordinary discussion; artifact management (RM9) as a defined folder structure with pointers to artifacts held in external systems; provenance linkage (RM10) as mandatory cross-referencing that assembles the question-to-output chain by hand, with no lineage query; reproducibility (RM12) as pinned versions plus a re-run workflow, with environment capture and archival left to external tools; and output identification (RM14) as tagged Releases plus `CITATION.cff`, with persistent identifiers supplied by the GitHub–Zenodo bridge. The two Limited requirements are the differentiator RM2 — no research-question object exists, so registration, revision history and question-to-artifact links are entirely manual and poorly discoverable — and RM15, where access control and contribution guidelines are handled well but ownership continuity, funding and workforce are not platform matters.

3.3 Reference architecture, template and feasibility (RQ4, RQ5)

Architecture (RQ4).

The covered capabilities organise into a **15-component, five-layer architecture** (Figure 2) in which GitHub is a coordination and traceability layer, not a container: coordination (research plan, activity tracker, question register, discussion space), record (documentation set, decision log, linkage discipline, change history), production (artifact workspace, review workflow, automation suite), release (release process, archival bridge, governance setup) and a cross-cutting transparency surface. Every requirement is served by at least one component. One component — *linkage discipline* (mandatory cross-referencing of every commit, pull request and issue) — and four

Table 2 Research management requirements (RQ2) and their GitHub support level (RQ3). Support: D Direct, P Partial, L Limited. ⋆ differentiator (high importance, weak literature attention).

ID	Requirement	Category	Support
RM1⋆	Research planning and roadmap	Planning	P
RM2⋆	Research question management	Traceability	L
RM3	Task management	Execution	D
RM4	Documentation	Documentation	D
RM5⋆	Decision traceability	Traceability	P
RM6	Collaboration and contribution tracking	Collaboration	D
RM7	Communication	Collaboration	D
RM8	Version control	Traceability	D
RM9	Research artifact management and integration	Artifact mgmt	P
RM10	Research provenance and artifact linkage	Traceability	P
RM11	Transparency	Documentation	D
RM12	Reproducibility support	Documentation	P
RM13	Automation	Automation	D
RM14	Research output management and identification	Output mgmt	P
RM15	Governance and sustainability	Cross-cutting	L

Table 3 GitHub coverage by requirement category and for the differentiators (RQ3). Mean support on a 0–3 scale.

Grouping	n	Mean support
Execution / Collaboration / Automation	4	3.00
Documentation	3	2.67
Planning / Traceability / Artifact mgmt / Output mgmt	7	2.00
Governance	1	1.00
Differentiators (RM1, RM2, RM5)	3	**1.67**
All other requirements	12	2.50
Overall (core 14)	14	2.43
Overall (full 15)	15	2.33
Overall, stricter rubric	15	2.13

named conventions (a phased plan of record, a question register, a decision-record convention and the linkage discipline itself) are the framework's contribution over a plain repository: they are what lift RM1, RM2, RM5 and RM10 from raw features toward research-usable support. The end-to-end traceability path they produce is shown in Figure 3. Seven workflow definitions specify the concrete step sequences (registering a research question, executing a task, recording a decision, reviewing an artifact, releasing and archiving, checking reproducibility, onboarding).

Template (RQ4).

The architecture is operationalised as a **33-file discipline-independent template**: a repository skeleton, a documentation set that states the four conventions, three issue

forms, a pull-request checklist, a label set, Project field definitions, starter automation and a release checklist. A manifest maps every file to the components and requirements it serves; all fifteen requirements are covered.

Feasibility (RQ5).

In the self-referential implementation, **13 of 15 components were exercised** natively or partially; only the discussion space and the Zenodo archival bridge remained planned. The record layer (23 documentation files, 26 logged decisions each with context, rationale, alternatives and affected artifacts), the artifact workspace and version control were realised at full strength from project start. After the repository was made public and the study's backlog was taken through one full Issue → pull request → Release cycle, the coordination layer (Issues, Project, Pull Requests) moved to observed use. The six-dimension evaluation (Table 4) then scores **1.86 of 2 overall** (1.85 for the twenty observed sub-scores): requirement coverage, documentation, organization and transparency reach 2.0, and usability is lowest at 1.50 — driven by configuration complexity and the assumed Git, Markdown and YAML literacy, a real barrier outside computational fields. Against a fragmented workflow (Table 5), the framework's benefit is concentrated in linking artifacts that would otherwise stay disconnected — decision rationale, version history and the question-to-result chain become reconstructable — and its cost is concentrated in the coordination layer, whose multi-author value a single-author case cannot establish.

External validity (RQ5).

Instantiating the template retrospectively on an unrelated published clinical-machine-learning project (mortality prediction in cardiac surgery, with model-interpretability analysis and a decision-support application) reproduced the self-referential coverage profile: 8 of the 15 requirements are served by a native template mechanism, 3 by an available-but-unexercised convention, and 4 — restricted patient data, environment capture, full reproduction and institutional governance — depend on infrastructure outside the repository. The documentation, decision, version-control and output spine transferred directly; the boundary fell in the same place, with the added constraint that clinical data is access-restricted rather than merely large. The check is retrospective, single-case and shares its lead author, so it tests structural transfer, not usability.

4 Discussion

4.1 Principal findings

The study's central result is a convergence. The gap identified bibliometrically in RQ1 — the literature under-serves idea and question formulation, planning and decision traceability — and the gap measured in RQ3 — GitHub supports those same requirements at 1.67 against 2.50 for the rest — are the same gap, reached by two independent routes. GitHub covers the execution-to-output span of the research lifecycle well and the upstream, research-specific layer poorly; the partiality is systematic, not incidental.

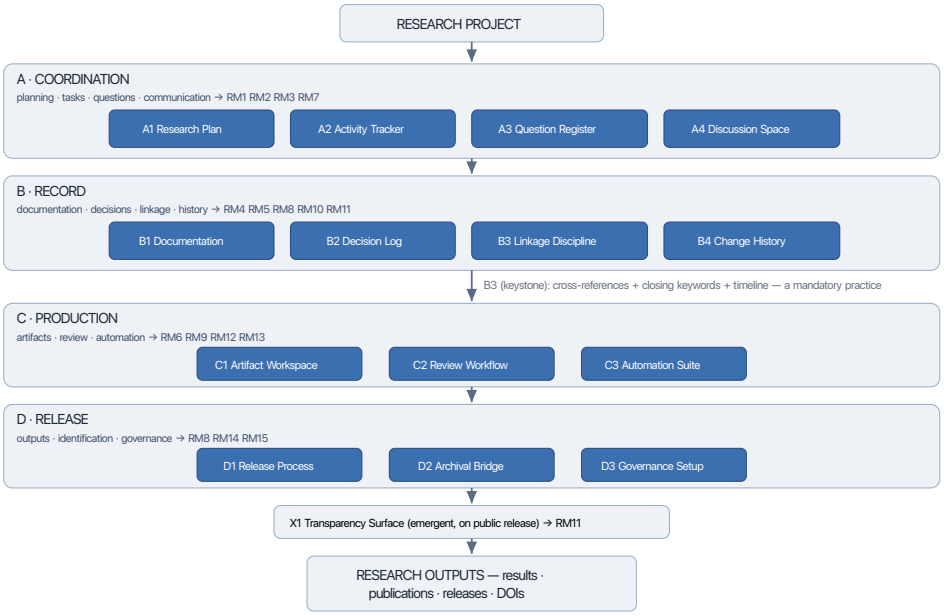

Fig. 2 GitHub-based research management reference architecture (RQ4): five layers, fifteen components. GitHub acts as a coordination and traceability layer; large data, computational environments, persistent identifiers and institutional governance remain external and are linked by reference.

4.2 GitHub as a coordination and traceability layer

Where research work can be expressed as files under version control, discrete tasks, branch-and-review changes and event-triggered automation, GitHub offers mature, purpose-built support, and the self-referential implementation confirms that this layer is low-cost: a full plan of record, twenty-six logged decisions and a reproducible analysis pipeline were produced with negligible overhead on the record layer, and the one full issue-to-release cycle run over the study's own backlog added only modest per-item effort for a solo author. Where the object of management is the evolving intellectual process — the questions asked, the choices made and why — GitHub offers only generic features on which a discipline must be imposed. The framework's contribution is to name that discipline: a mandatory linkage convention and three record conventions that make a plain repository into a traceable one. Even so, the differentiator requirements remain Partial or Limited; the framework maps the gap rather than closing it.

4.3 Which lifecycle stages benefit

Both the literature and GitHub's coverage are weakest at the earliest stages. The execution-to-output span — methods, data, analysis, results, manuscript, release —

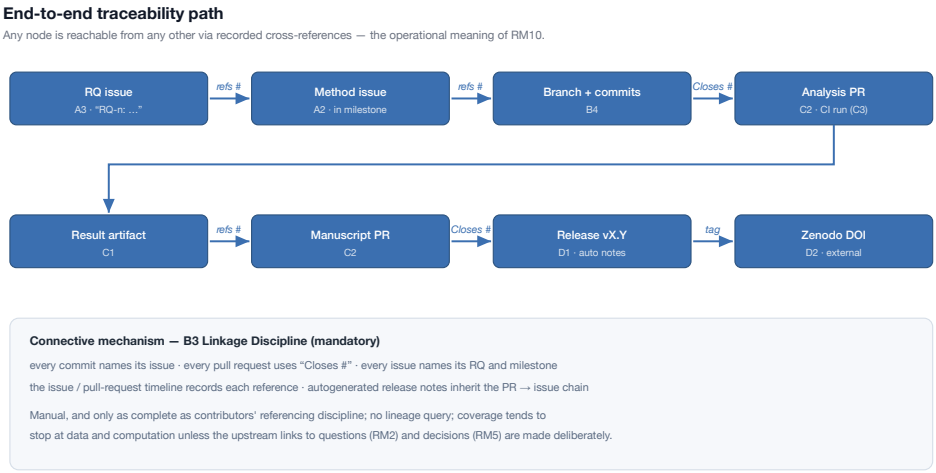

Fig. 3 End-to-end traceability path produced by the linkage-discipline convention: a research-question issue is referenced by the method issue, whose branch and commits feed a reviewed pull request, whose result feeds the manuscript and a tagged release archived with a DOI. Any node is reachable from any other via recorded cross-references.

Table 4 Framework evaluation on six dimensions (RQ5), 0–2 scale, from the self-referential case study, after the repository was made public and one full issue-to-release cycle was run. "Observed" marks sub-scores evidenced by the actual repository state.

Dimension	Basis	Score
E1 Requirement coverage	design-support	2.00
E2 Traceability	observed	1.75
E3 Documentation	observed	2.00
E4 Organization	observed	2.00
E5 Transparency	observed	2.00
E6 Usability	observed	1.50
Overall (21 sub-scores)		**1.86**
Overall, observed only (20)		1.85

benefits most, because its artifacts are files and its activities are changes. The idea, question and planning stages benefit least, because they are deliberative and their "artifacts" are decisions. This is where tool development and reporting standards — for decision records and research-question provenance — would have the most leverage.

Table 5 Fragmented workflow versus the GitHub-based framework (RQ5).

Dimension	Fragmented workflow	GitHub-based framework
Task traceability	tasks disconnected from the artifacts they produce	issues plus board plus closing keywords bind task to commit to pull request
Decision traceability	decisions in email and notes, rationale lost	numbered decision log plus decision issue, with rationale and affected artifacts
Version traceability	files renamed "final"; no history	Git line-level history, diff, restore, tags
Documentation	scattered documents; drift; no history	version-controlled Markdown from project start, with per-folder READMEs
Research provenance	no systematic link between question, data and result	cross-references and timelines assemble the chain; release notes inherit it
Project visibility	known only to the email loop	public repository plus status, activity log and changelog

4.4 Practical implications

For an individual researcher or a small group, the practical recommendation is to adopt the record and version-control layer first: a version-controlled `docs` directory, a decision log and disciplined commit messages deliver most of the traceability benefit at little cost. The Issue, Project and Pull Request layer should be added when the project has collaborators, where its coordination value accrues and its overhead is justified. The 33-file template lowers the setup cost of both.

4.5 Relationship with output-preservation workflows

This study addresses the management of the research process; a complementary line of work addresses the preservation and persistent identification of research *outputs* through GitHub, Zenodo and ORCID. The two are the upstream and downstream halves of one lifecycle: the framework's release and archival-bridge components hand a versioned, cross-referenced record to an external archive that assigns a DOI. GitHub-based process management extends, rather than replaces, the open-science tooling around outputs.

4.6 Limitations

Six limitations bound the findings. First, the framework maps the upstream gap but does not close it: RM1, RM2 and RM5 stay Partial or Limited even with the prescribed conventions. Second, the coordination layer's *collaborative* benefit is unproven: it was run through once end to end, but under single-author conditions, so the multi-author value of RM6, RM7 and the Issue/Project/Pull Request layer — review, communication, parallel work — was not observed. Third, the evaluation is single-case and

self-referential, and every coding step had a single human coder; an independent recode of all 53 coded decisions agreed at 96% (κ 0.83–1.00) and left the reported figures unchanged, but a second independent human rater was not available, and the retrospective second-project instantiation shares this study's lead author. Fourth, the primary bibliometric corpus is OpenAlex plus arXiv; a Scopus and Web of Science cross-check (18,250 raw records) shares 22% of its DOIs with the corpus, the divergence being a characterised precision tail of tool-citing papers rather than missed core literature, and the broadest-query precision is ≈ 0.94. Fifth, roughly a third of the coverage rests on mandatory convention rather than native features, as the stricter-rubric mean of 2.13 shows. Sixth, GitHub is not a complete research infrastructure: persistent identifiers and preservation, computational environments, large datasets and institutional governance are all external.

4.7 Future research

Three directions follow. The retrospective, single-author template instantiation reported here establishes structural transfer to another discipline; a *prospective*, multi-author instantiation on a project run from the start would test the coordination layer's collaborative value and usability under realistic conditions. Instrumenting the linkage-discipline convention — automatically checking that commits, pull requests and issues cross-reference their research question and decisions — would move provenance from a manual practice toward a verifiable one. And a bibliometric follow-up on the upstream stages specifically would quantify whether the idea/question/planning gap is beginning to close as AI-assisted planning and digital ethics platforms mature.

5 Conclusions

GitHub can serve as an integrated coordination and traceability *layer* for a substantial part of the scientific research lifecycle. It supports thirteen of fifteen literature-derived research management requirements at partial or better and seven directly, with none unsupported; it is strong from execution to output and for research that resembles software engineering; and it is weak for the upstream, research-specific traceability of questions, decisions and provenance — the very requirements the literature also under-serves. Five requirements need complementary external infrastructure. A fifteen-component reference architecture, four mandatory conventions and a thirty-three-file template operationalise what GitHub covers, and a self-referential implementation shows that its documentation and version-control layer works at near-zero overhead while the collaborative value of its coordination layer remains to be demonstrated. GitHub maps the research-process-management gap rather than closing it, and it is best understood not as a replacement for research infrastructure but as the coordination and traceability layer that a fragmented set of research tools currently lacks.

Declarations

Data and code availability

The study repository — bibliographic corpus, coding sheets, requirement and mapping tables, reference architecture, reusable template and all analysis scripts — is openly available at https://github.com/telmomm/github-research-infrastructure-study and archived on Zenodo, DOI 10.5281/zenodo.22167500 (software record, a new version deposited per release). A companion dataset record, DOI 10.5281/zenodo.22173525, holds the Track A bibliometric corpus, the processed datasets and the raw database retrieval exports. Every table and figure in this paper is regenerated from these inputs by a versioned script.

Competing interests

The author declares no competing interests.

Funding

This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Ethics approval

Not applicable. The study involves no human participants, human data or animals; the bibliographic corpus consists of publication metadata from public indexes.

Author contributions

T.M.-M. conceived and designed the study, performed the analysis, developed the framework and template, and wrote the manuscript.

References

[1] Liew, C.S., Atkinson, M.P., Galea, M., Ang, T.F., Martin, P., Hemert, J.I.: Scientific workflows: Moving across paradigms. ACM Computing Surveys **49**(4), 1–39 (2016) https://doi.org/10.1145/3012429

[2] Liu, J., Pacitti, E., Valduriez, P., Mattoso, M.: A survey of data-intensive scientific workflow management. Journal of Grid Computing **13**(4), 457–493 (2015) https://doi.org/10.1007/s10723-015-9329-8

[3] Mattoso, M., Dias, J., Ocaña, K.A.C.S., Ogasawara, E., Costa, F., Horta, F., Sousa, V., Oliveira, D.: Dynamic steering of HPC scientific workflows: A survey. Future Generation Computer Systems **46**, 100–113 (2015) https://doi.org/10.1016/j.future.2014.11.017

[4] Perrier, L., Blondal, E., Ayala, A.P., Dearborn, D., Kenny, T., Lightfoot, D., Reka, R., Thuna, M., Trimble, L., MacDonald, H.: Research data management

in academic institutions: A scoping review. PLoS ONE **12**(5), 0178261 (2017) https://doi.org/10.1371/journal.pone.0178261

[5] Donner, E.K.: Research data management systems and the organization of universities and research institutes: A systematic literature review. Journal of Librarianship and Information Science **55**(2), 261–281 (2022) https://doi.org/10.1177/09610006211070282

[6] Crane, K., Blatch-Jones, A., Fackrell, K.: The post-award effort of managing and reporting on funded research: a scoping review. F1000Research **12**, 863 (2023) https://doi.org/10.12688/f1000research.133263.2

[7] Tutko, A., Henley, A.Z., Mockus, A.: How are software repositories mined? a systematic literature review of workflows, methodologies, reproducibility, and tools. arXiv **abs/2204.08108** (2022) https://doi.org/10.48550/arxiv.2204.08108

[8] Davidson, S.B., Freire, J.: Provenance and scientific workflows: challenges and opportunities. In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, pp. 1345–1350 (2008). https://doi.org/10.1145/1376616.1376772

[9] Pérez, B., Rubio, J., Sáenz-Adán, C.: A systematic review of provenance systems. Knowledge and Information Systems **57**(3), 495–543 (2018) https://doi.org/10.1007/s10115-018-1164-3

[10] Ho, R.C.Y., Wong, S.N., Chia, P., Tang, C., Ng, M.: Research data management services in academic libraries to support the research data life cycle: A systematic review. an annual review of information science and technology (ARIST) paper. Journal of the Association for Information Science and Technology **77**(1), 272–300 (2025) https://doi.org/10.1002/asi.70008

[11] Kirkham, J.J., Penfold, N.C., Murphy, F., Boutron, I., Ioannidis, J.P., Polka, J.K., Moher, D.: Systematic examination of preprint platforms for use in the medical and biomedical sciences setting. BMJ Open **10**(12), 041849 (2020) https://doi.org/10.1136/bmjopen-2020-041849

[12] Mayernik, M.S., Hart, D.L., Maull, K.E., Weber, N.M.: Assessing and tracing the outcomes and impact of research infrastructures. Journal of the Association for Information Science and Technology **68**(6), 1341–1359 (2017) https://doi.org/10.1002/asi.23721

[13] Cox, A., Tam, W.: A critical analysis of lifecycle models of the research process and research data management. Aslib Journal of Information Management **70**(2), 142–157 (2018) https://doi.org/10.1108/ajim-11-2017-0251

[14] Holler, J., Kedron, P.: Mainstreaming metadata into research workflows to

advance reproducibility and open geographic information science. In: The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, vol. XLVIII-4/W1-2022, pp. 201–208 (2022). https://doi.org/10.5194/isprs-archives-xlviii-4-w1-2022-201-2022

[15] Devkota, N., Siddique, M., Karki, D., Thapa, D.: Research in the age of AI — how to plan? a bibliometric exploration of the research lifecycle. International Research Journal of MMC **7**(3), 94–116 (2026) https://doi.org/10.3126/irjmmc.v7i3.97169

[16] Oliveira, W., Oliveira, D., Braganholo, V.: Provenance analytics for workflow-based computational experiments: A survey. ACM Computing Surveys **51**(3), 1–25 (2018) https://doi.org/10.1145/3184900

[17] Haris, M., Auer, S., Stocker, M.: Research information systems and knowledge graphs: a review. Frontiers in Research Metrics and Analytics **11**, 1786866 (2026) https://doi.org/10.3389/frma.2026.1786866

[18] Klebel, T., Traag, V., Grypari, I., Stoy, L., Ross-Hellauer, T.: The academic impact of open science: a scoping review. Royal Society Open Science **12**(3), 241248 (2025) https://doi.org/10.1098/rsos.241248

[19] Sandve, G.K., Nekrutenko, A., Taylor, J., Hovig, E.: Ten simple rules for reproducible computational research. PLoS Computational Biology **9**(10), 1003285 (2013) https://doi.org/10.1371/journal.pcbi.1003285

[20] Blischak, J.D., Davenport, E.R., Wilson, G.: A quick introduction to version control with Git and GitHub. PLoS Computational Biology **12**(1), 1004668 (2016) https://doi.org/10.1371/journal.pcbi.1004668

[21] Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., Teal, T.K.: Good enough practices in scientific computing. PLoS Computational Biology **13**(6), 1005510 (2017) https://doi.org/10.1371/journal.pcbi.1005510

[22] Braga, P.H.P., Hébert, K., Hudgins, E.J., Scott, E.R., Edwards, B.P.M., Sylvain, J.D., Beaubien, S., Bouchard, F., Colwill, S., Domke, S., Fisher, J.A.D., Hunter, F.F., Prokopiuk, M.: Not just for programmers: How GitHub can accelerate collaborative and reproducible research in ecology and evolution. Methods in Ecology and Evolution **14**(6), 1364–1380 (2023) https://doi.org/10.1111/2041-210X.14108

[23] Pérez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Veiga Leprevost, F., Fufezan, C., Ternent, T., Eglen, S.J., Katz, D.S., Pollard, T.J., Konovalov, A., Flight, R.M., Blin, K., Vizcaíno, J.A.: Ten simple rules for taking advantage of Git and GitHub. PLoS Computational Biology **12**(7), 1004947 (2016) https://doi.org/10.1371/journal.pcbi.1004947

[24] Chen, K.M., Toro-Moreno, M., Subramaniam, A.R.: GitHub enables collaborative

and reproducible laboratory research. PLoS Biology **23**(2), 3003029 (2025) https://doi.org/10.1371/journal.pbio.3003029

[25] Crystal-Ornelas, R., Varadharajan, C., Bond-Lamberty, B., Boye, K., Burrus, M., Cholia, S., Crow, M., Damerow, J., Devarakonda, R., Ely, K., Goldman, A., Heinz, S., Hendrix, V., Kakalia, Z., Pennington, S., Robles, E., Rogers, A., Simmonds, M., Velliquette, T., Agarwal, D.: A guide to using GitHub for developing and versioning data standards and reporting formats. Earth and Space Science **8**(8), 2021–001797 (2021) https://doi.org/10.1029/2021ea001797

[26] Escamilla, E., Klein, M., Cooper, T., Rampin, V., Weigle, M.C., Nelson, M.L.: The rise of GitHub in scholarly publications. In: Linking Theory and Practice of Digital Libraries (TPDL 2022). Lecture Notes in Computer Science, vol. 13541, pp. 187–200 (2022). https://doi.org/10.1007/978-3-031-16802-4_15

[27] Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D.: An in-depth study of the promises and perils of mining GitHub. Empirical Software Engineering **21**(5), 2035–2071 (2016) https://doi.org/10.1007/s10664-015-9393-5

[28] Trisovic, A., Lau, M.K., Pasquier, T., Crosas, M.: A large-scale study on research code quality and execution. Scientific Data **9**(1), 60 (2022) https://doi.org/10.1038/s41597-022-01143-6